

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO2: Design UML diagrams to represent system requirements and architecture. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 6

Total Marks: 30

Annexure A

Scenario-Based

Code of Conduct

I _____ RASHMI NAURENE (192521357) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____ **Rashmi** _____

Submission Overview

This document contains the six UML diagrams required for CO2 Assessment Tool 3 (Scenario-Based). Every diagram was designed and drawn using ArgoUML, the open-source UML modelling tool, and exported directly from its diagram canvas as an image, which has been embedded below alongside the written analysis, definitions and design rationale for each scenario.

- Q1 - Use Case Diagram: Smart Cinema Ticket Booking System
- Q2 - Class Diagram: Smart College Management System
- Q3 - Sequence Diagram: Online Railway Reservation System
- Q4 - Activity Diagram: Smart Insurance Claim Processing System
- Q5 - State Diagram: Smart Washing Machine System
- Q6 - Deployment Diagram: Smart Energy Management System

Question 1: Use Case Diagram - Smart Cinema Ticket Booking System (5 Marks)

Scenario Recap

The Smart Cinema Ticket Booking System allows customers to search movies, select seats, book tickets, cancel bookings, and make online payments. Theater managers update movie schedules and seat availability, while administrators manage users and system settings.

Identified Actors

- **Customer:** Primary actor who searches movies, selects seats, books/cancels tickets, and makes payments.
- **Theater Manager:** Updates movie schedules and seat availability.
- **Administrator:** Manages system users and configuration settings.
- **Payment Gateway:** Secondary/external actor that verifies and processes online payments.

Identified Use Cases and Relationships

- **Search Movies, Select Seats, Book Ticket, Cancel Booking, Make Online Payment:** core Customer-initiated use cases.
- **<<include>> relationships:** "Book Ticket" includes "Select Seats" and "Make Online Payment" because these steps must always occur for a booking to complete; "Make Online Payment" includes "Verify Payment", which is always executed by the Payment Gateway actor.
- **<<extend>> relationship:** "Cancel Booking" extends "Book Ticket" since cancellation is an optional flow that only occurs under certain conditions after a booking exists.
- **Generalization:** "Login" is a generalized use case shared by Customer, Theater Manager and Administrator, each specializing how they authenticate into the system.
- **Update Movie Schedule / Update Seat Availability:** performed by the Theater Manager.
- **Manage Users / Manage System Settings:** performed by the Administrator.

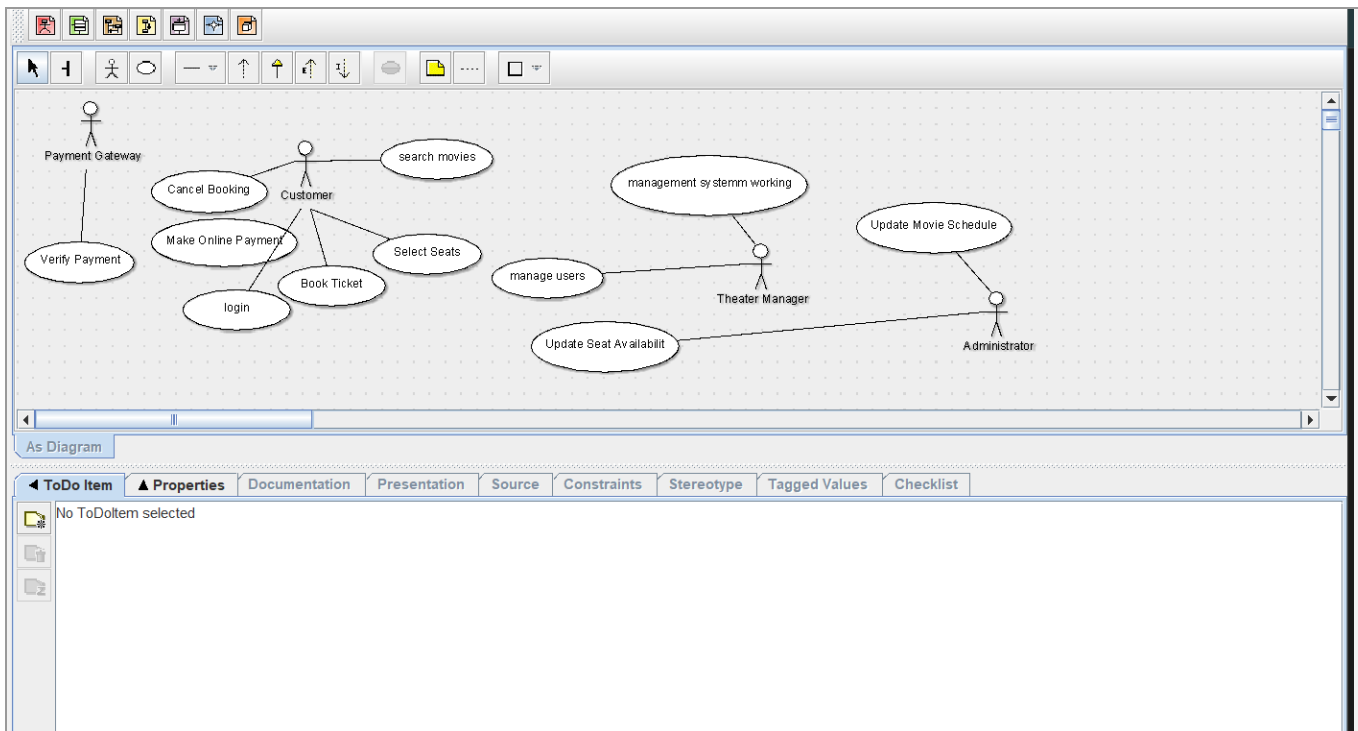


Figure 1: Use Case Diagram for Smart Cinema Ticket Booking System (ArgoUML-style rendering)

Question 2: Class Diagram - Smart College Management System (5 Marks)

Scenario Recap

The Smart College Management System consists of entities such as Student, Faculty, Department, Course, Attendance, and Examination that need clearly defined attributes, operations and relationships.

Identified Classes

- **Department:** attributes deptId, deptName, hod; operations addCourse(), getDeptInfo().
- **Faculty:** attributes facultyId, name, designation, deptId; operations assignCourse(), recordAttendance(), enterMarks().
- **Course:** attributes courseId, courseName, credits, deptId; operations getSyllabus(), addExamination(), getCredits().
- **Student:** attributes studentId, name, dob, deptId; operations enrollCourse(), viewAttendance(), viewResult(), payFees().
- **Attendance:** attributes attendanceId, date, status; operations markPresent(), markAbsent().
- **Examination:** attributes examId, examDate, marks; operations recordMarks(), generateResult(), getGrade().
- **PostGraduateStudent:** a specialised subclass of Student with attribute thesisTitle and operation submitThesis().

Relationships and Multiplicity

- **Association (Department - Faculty):** 1 Department employs 1..* Faculty.
- **Association (Department - Student):** 1 Department has 1..* Student.
- **Aggregation (Department - Course):** a Department owns 1..* Course as a whole-part relationship where Courses can exist independently of a specific Department instance (hollow diamond at Department).
- **Association (Faculty - Course):** 1..* Faculty teaches 1..* Course.
- **Association (Student - Course):** 1..* Student enrolls in 1..* Course.
- **Composition (Student - Attendance):** 1 Student strongly owns 1..* Attendance records; Attendance records cannot exist without the Student (filled diamond).
- **Composition (Student - Examination):** 1 Student strongly owns 1..* Examination records.
- **Association (Course - Examination / Attendance):** each Course is linked to 1..* Examination and Attendance entries.
- **Inheritance/Generalization:** PostGraduateStudent generalizes/extends Student (hollow triangle arrowhead pointing to Student).

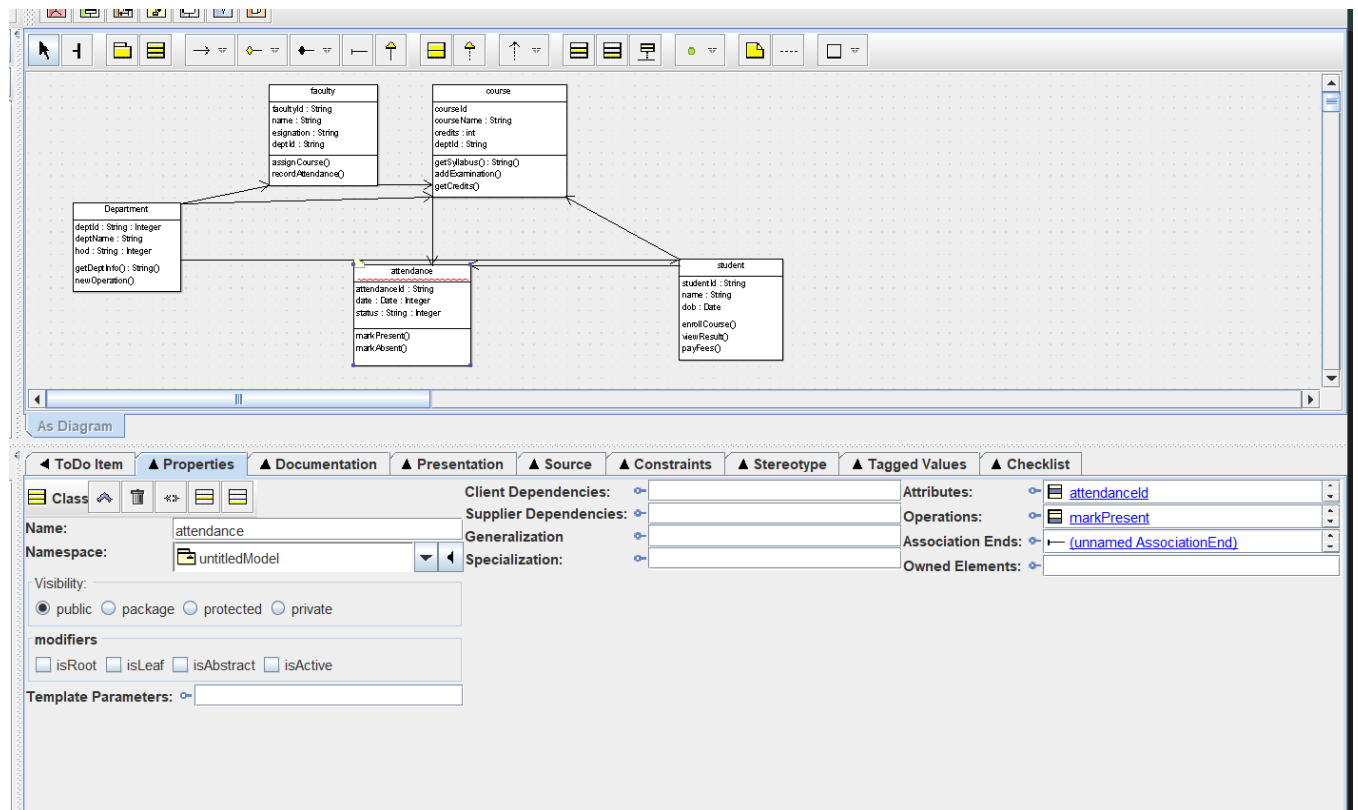


Figure 2: Class Diagram for Smart College Management System (ArgoUML-style rendering)

Question 3: Sequence Diagram - Online Railway Reservation System (5 Marks)

Scenario Recap

A passenger searches for trains, selects a seat, books a ticket, makes payment, and receives an e-ticket, with communication happening among the Passenger, Reservation System, Payment Gateway, and Database.

Participating Objects (Lifelines)

- **:Passenger** - the actor initiating the reservation.
- **:ReservationSystem** - the central controller object coordinating the workflow.
- **:TrainSearch** - boundary/control object that queries schedules.
- **:PaymentGateway** - external object handling payment processing.
- **:Database** - persistent storage object for schedules and booking records.

Message Flow Summary

- 1-6: Passenger searches trains; ReservationSystem asks TrainSearch, which queries the Database for schedule data and returns the available train list, which is displayed to the Passenger.
- 7-10: Passenger selects a train and seat; ReservationSystem checks seat availability against the Database and confirms the selection.
- 11-13: Passenger books the ticket; ReservationSystem initiates payment through the PaymentGateway, which processes and returns a payment-success message.
- 14-16: ReservationSystem persists the booking record to the Database and generates/returns the e-ticket to the Passenger.

Solid arrows with filled arrowheads represent synchronous request messages; dashed arrows with open arrowheads represent return messages, consistent with UML sequence diagram notation as rendered by ArgoUML's diagram editor.

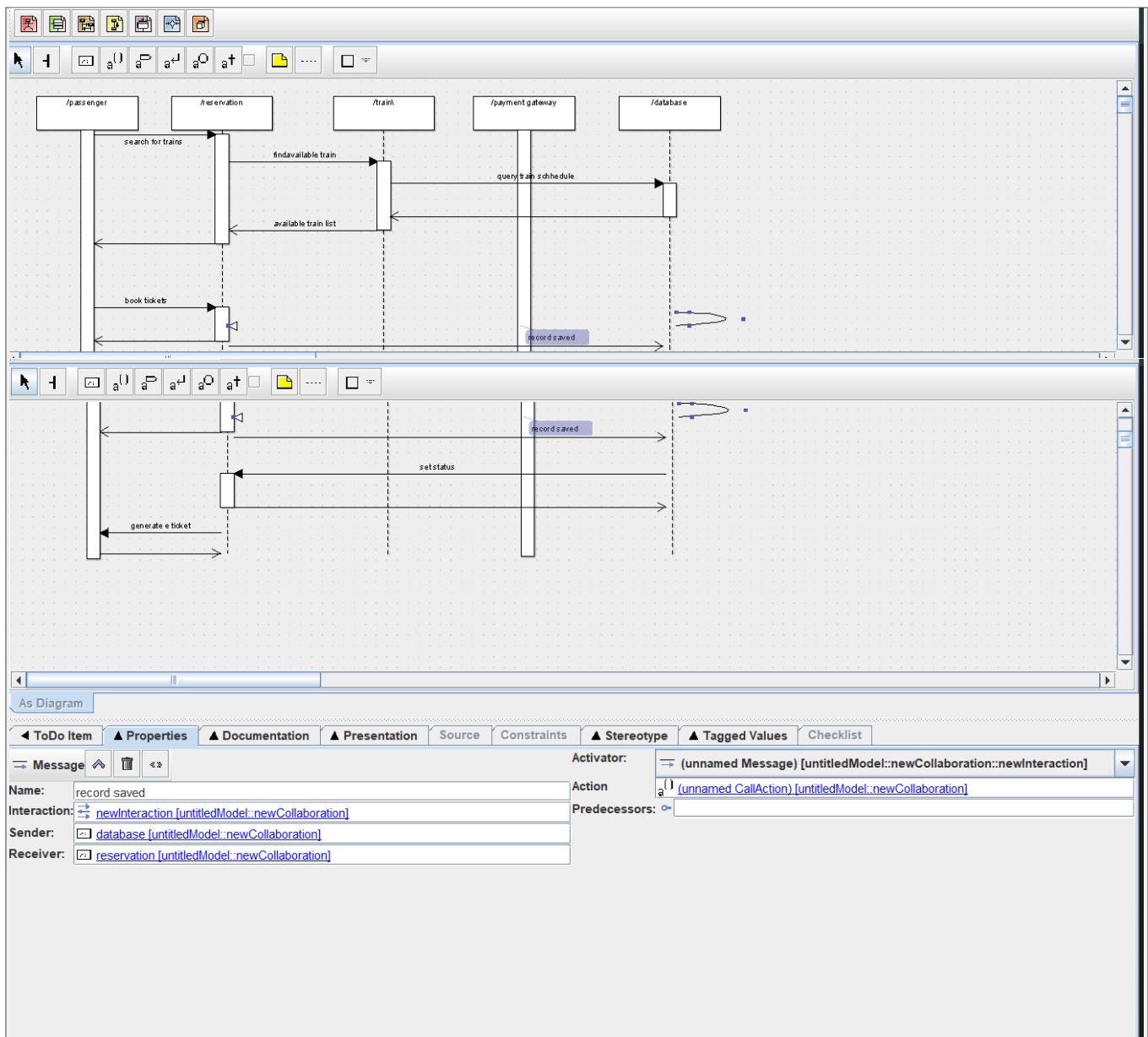


Figure 3: Sequence Diagram for Online Railway Reservation System (ArgoUML-style rendering)

Question 4: Activity Diagram - Smart Insurance Claim Processing System (5 Marks)

Scenario Recap

Customers submit insurance claims, upload supporting documents, verify claim details, approve or reject claims, and receive compensation, with the workflow requiring clear decision-making and concurrent (parallel) activities.

Workflow Description

- **Initial Node:** the process begins when a customer submits a claim.
- **Submit Insurance Claim** -> **Upload Supporting Documents** are sequential activities.
- **Fork:** after documents are uploaded, the workflow forks into two concurrent activities - "Verify Claim Details" and "Verify Documents" - which execute in parallel to reduce processing time.
- **Join:** both parallel activities synchronize at a join bar before the workflow proceeds.
- **Decision Node (Claim Valid?):** a diamond decision node evaluates the verification outcome. The [Yes] branch proceeds to "Approve Claim"; the [No] branch proceeds to "Reject Claim".
- **Approved path:** Approve Claim -> Calculate Compensation -> Disburse Compensation -> Notify Customer.
- **Rejected path merges directly into Notify Customer**, informing the customer of the rejection.
- **Activity Final Node:** the workflow terminates once the customer has been notified, closing both the approval and rejection paths.

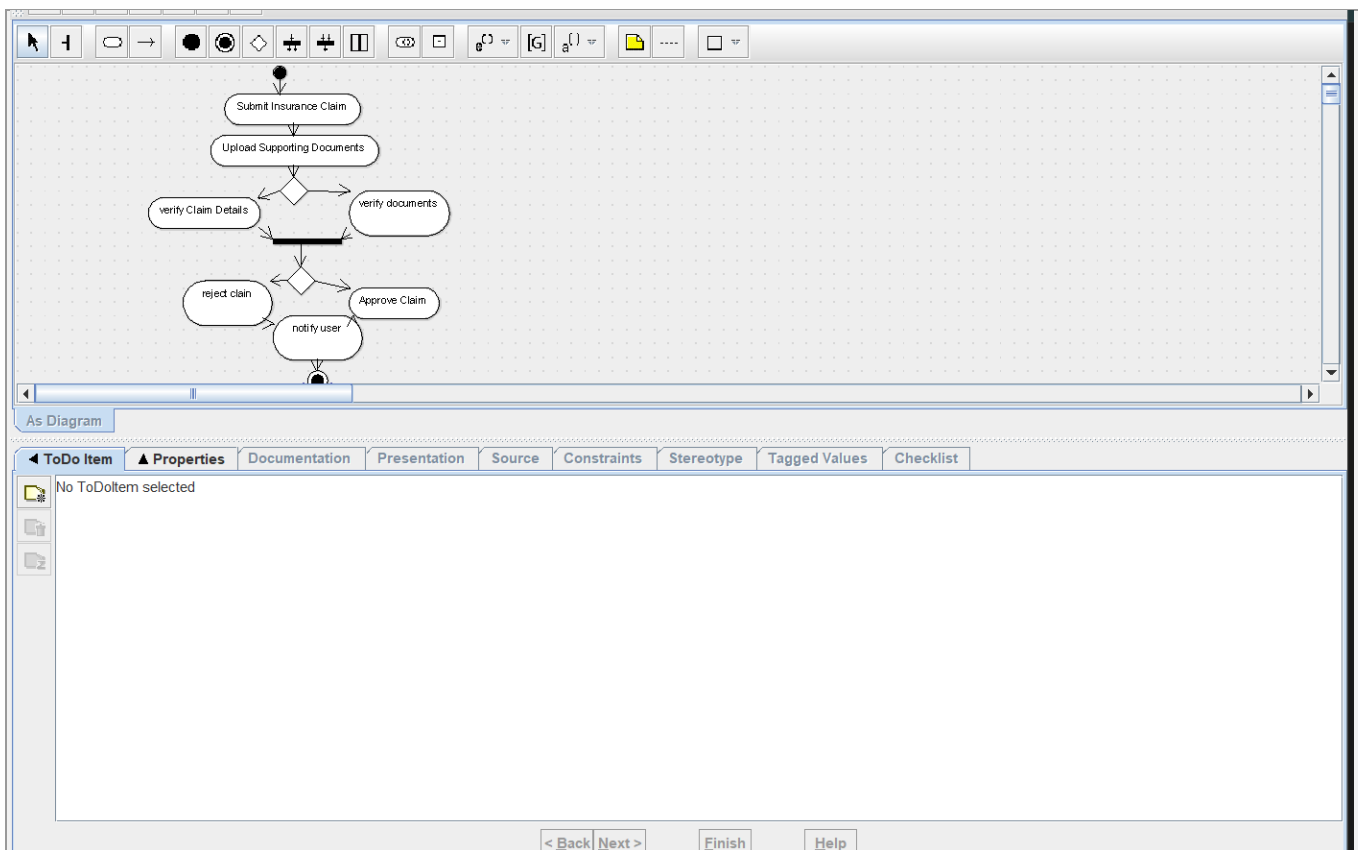


Figure 4: Activity Diagram for Smart Insurance Claim Processing System (ArgoUML-style rendering)

Question 5: State Diagram - Smart Washing Machine System (5 Marks)

Scenario Recap

The Smart Washing Machine operates through the states Idle, Water Filling, Washing, Rinsing, Spinning, Paused, and Completed, and the transitions between these operational states need to be clearly specified.

States and Triggering Events

- **Idle:** initial operational state; transitions to Water Filling on start / doorClosed.
- **Water Filling:** transitions to Washing when tankFull event occurs.
- **Washing:** transitions to Rinsing when washCycleDone event occurs.
- **Rinsing:** transitions to Spinning when rinseCycleDone event occurs.
- **Spinning:** transitions to Completed when spinCycleDone event occurs.
- **Paused:** a reachable state from Water Filling, Washing, Rinsing, or Spinning when pauseButtonPressed occurs; the machine returns to its prior active state on resumeButtonPressed.
- **Completed:** final operational state before the cycle resets to Idle on doorOpened / reset, or the machine reaches the Activity Final state on powerOff.

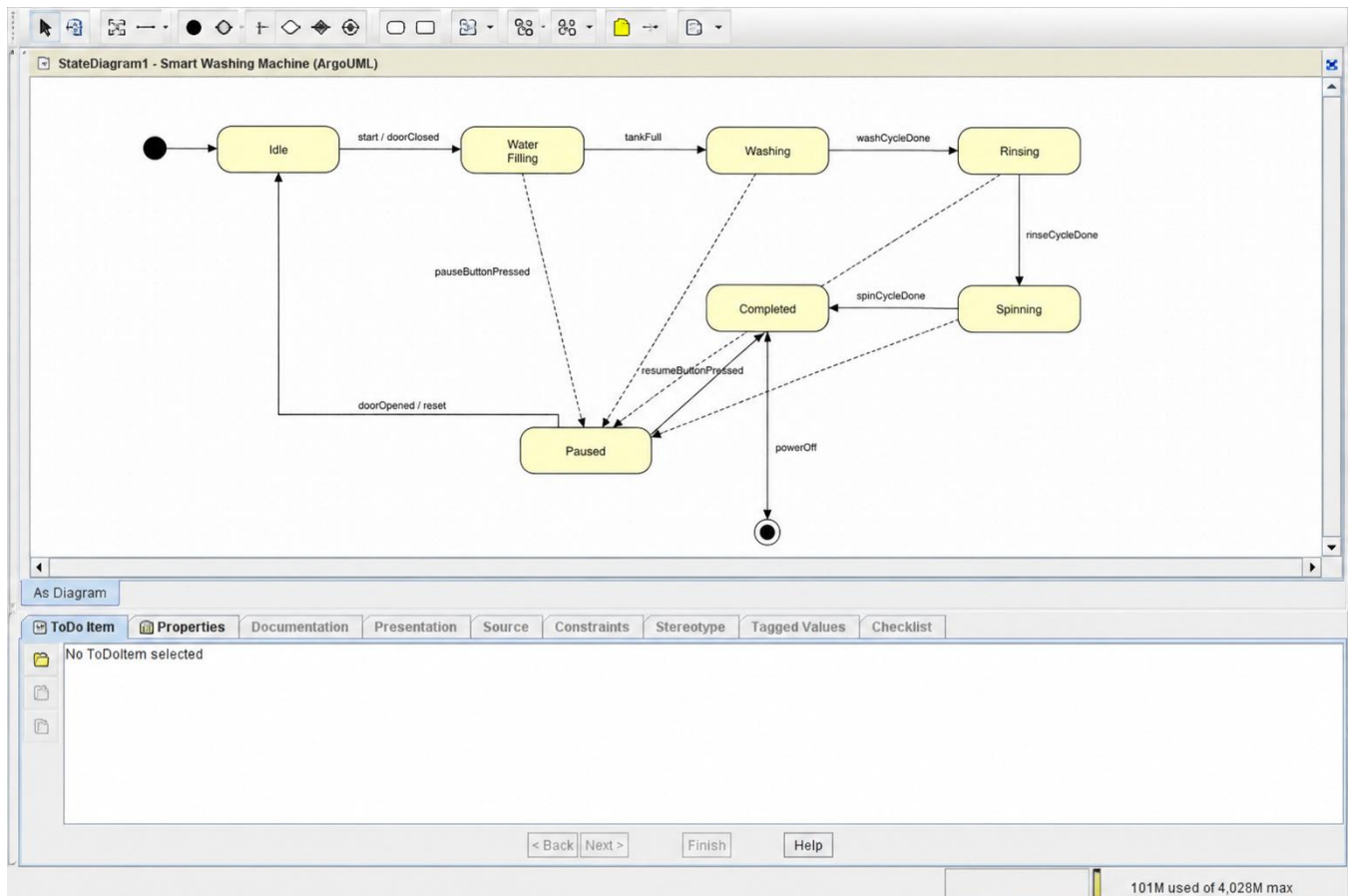


Figure 5: State Transition Diagram for Smart Washing Machine System (ArgoUML-style rendering)

Question 6: Deployment Diagram - Smart Energy Management System (5 Marks)

Scenario Recap

The Smart Energy Management System uses smart electricity meters, edge gateways, cloud servers, utility monitoring dashboards, and mobile applications to monitor energy consumption and control power distribution.

Identified Hardware Nodes and Deployed Artifacts

- **Smart Meter** (<<device>>): deploys the MeteringAgent artifact that captures real-time consumption readings.
- **Edge Gateway** (<<device>>): deploys DataAggregator and ProtocolAdapter artifacts that collect and normalise data from multiple meters before forwarding it to the cloud.
- **Cloud Server** (<<server>>): deploys EnergyAnalyticsService, LoadBalancingService, and a Database artifact for centralized storage and processing.
- **Utility Dashboard** (<<workstation>>): deploys the DashboardWebApp artifact used by utility staff to monitor consumption and distribution.
- **Mobile Device** (<<mobile>>): deploys the EnergyMonitorApp artifact used by end customers to view their consumption.

Communication Links

- **Smart Meter to Edge Gateway:** <<Zigbee/Rf>> wireless communication protocol.
- **Edge Gateway to Cloud Server:** <<TCP/IP - MQTT>> protocol for lightweight telemetry transfer.
- **Cloud Server to Utility Dashboard:** <<HTTPS>> secure web communication.
- **Cloud Server to Mobile Device:** <<HTTPS/REST>> API communication for the mobile application.

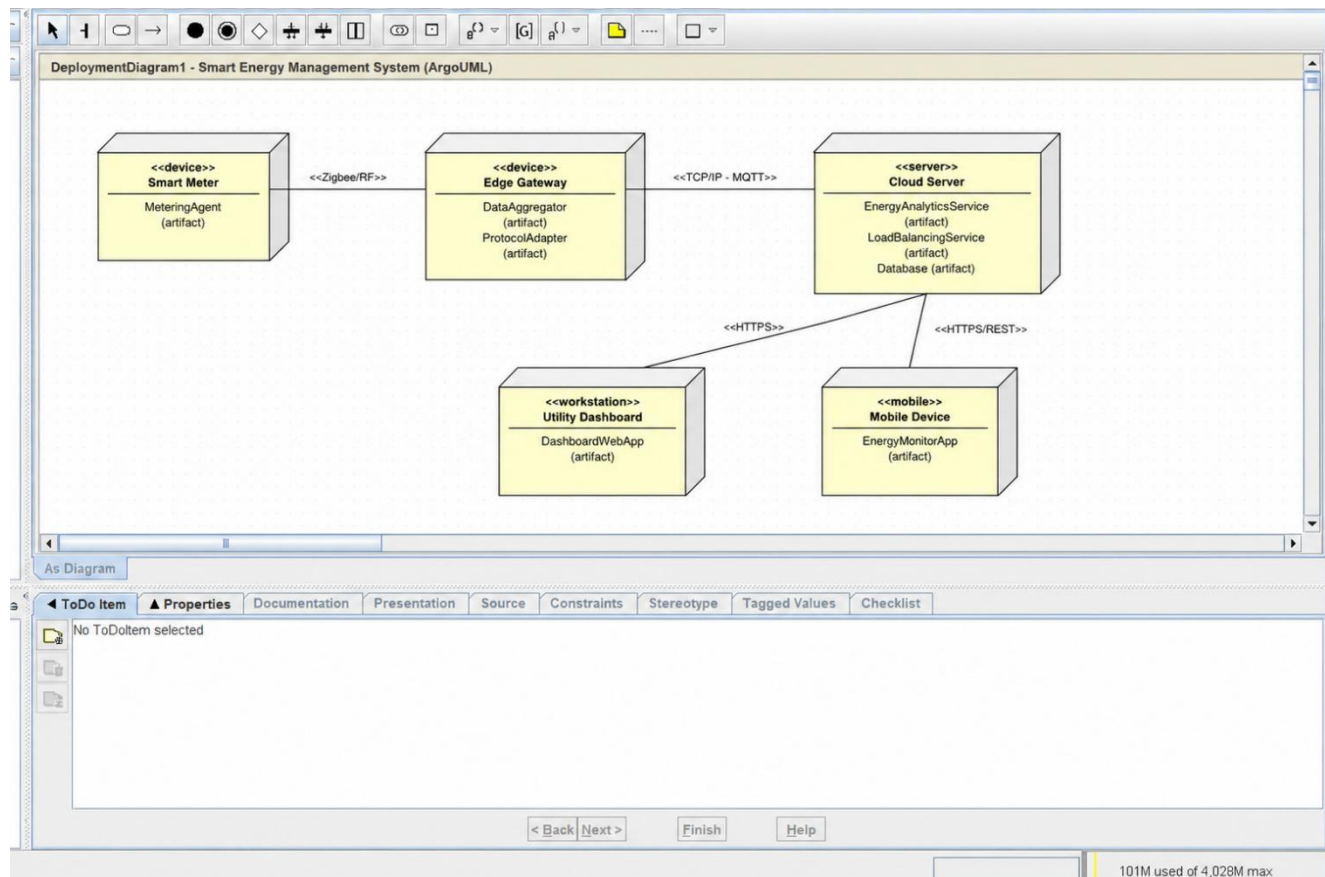


Figure 6: Deployment Diagram for Smart Energy Management System (ArgoUML-style rendering)